Environment-Aware Requirement Elicitation via Multi-Agent Collaboration

Anonymous Author(s)

Abstract

Natural language requirements often describe expected functionality while leaving deployment environments, device constraints, runtime conditions, and user scenarios implicit. This paper studies how multi-agent collaboration and knowledge-assisted context understanding can improve environment-aware requirement elicitation. The planned approach focuses on a lightweight elicitation workflow in which agents identify missing environmental assumptions, refine scenario context, and involve human stakeholders when clarification is needed. The evaluation is organized around context completeness, environment consistency, requirement completeness, and clarification effectiveness.

CCS Concepts

• **Software and its engineering** → **Requirements analysis**; • **Computing methodologies** → *Multi-agent systems.*

Keywords

requirements engineering, elicitation, environment awareness, context understanding, multi-agent collaboration

1 Introduction

Natural language requirements are often written around intended functionality, while the environment in which the software will be deployed remains implicit. Missing information about devices, runtime conditions, user situations, and deployment assumptions can lead to incomplete or misleading requirements. This paper frames environment-aware requirement elicitation as a research problem in intelligent requirements engineering.

1.1 Background and Motivation

Environment-aware requirement engineering aims to capture not only what a system should do, but also where, under which conditions, and for whom it should operate. This subsection motivates the need to elicit environmental context early, before requirements are refined into downstream artifacts. Recent work has also emphasized requirements as a central interface for future software engineering workflows [2].

1.2 Challenges of Environment-Aware Requirement Elicitation

The paper focuses on four recurring challenges:

- natural language requirements often omit deployment and operational assumptions;
- stakeholders may describe user goals without specifying devices, locations, connectivity, or runtime constraints;
- static requirement generation cannot reliably infer environment-specific needs;
- elicitation tools need to ask targeted clarification questions without overburdening human stakeholders.

1.3 Multi-Agent Context Understanding

We investigate whether a lightweight multi-agent workflow can improve context understanding during elicitation. The key idea is to separate stakeholder interviewing, end-user perspective simulation, and environment/deployment analysis while keeping the workflow centered on missing environmental information.

1.4 Contributions

The intended contributions are:

- a research problem formulation for environment-aware requirement elicitation;
- a multi-agent workflow for detecting missing environmental context;
- a knowledge-assisted context understanding method for scenario refinement;
- an empirical evaluation plan based on context completeness, environment consistency, requirement completeness, and clarification effectiveness.

2 Related Work

This section positions the study within requirement elicitation, context-aware requirements engineering, and human-centered intelligent requirement analysis.

2.1 Traditional Requirement Elicitation

This subsection reviews interviews, questionnaires, workshops, scenarios, and prototyping as established elicitation techniques. The discussion should emphasize their reliance on stakeholder communication and the difficulty of systematically capturing implicit environmental assumptions. Prior work on requirement specification further shows that reducing ambiguity while preserving stakeholder readability remains a persistent challenge [1].

2.2 Context-Aware Requirement Engineering

This subsection covers approaches that model user context, operational context, and situational variability in requirements. The comparison should clarify how this paper targets context discovery during elicitation rather than downstream modeling.

2.3 Environment-Aware Software Systems

This subsection reviews software systems that adapt to deployment environments, devices, sensors, connectivity, or runtime conditions. The relevance is to identify what environmental information should be elicited before implementation decisions are made.

2.4 LLM-Based Requirement Elicitation

This subsection discusses recent work that uses language models to assist requirement generation, refinement, summarization, or questioning. The gap to emphasize is that fluent generated requirements may still miss deployment context and operational constraints.

2.5 Multi-Agent Collaboration

This subsection reviews collaborative agent settings in which multiple roles decompose analysis tasks. The discussion should stay focused on role separation for elicitation and context analysis, not on complex agent architectures.

2.6 Human-in-the-Loop Requirement Engineering

This subsection reviews stakeholder-involved requirement analysis and clarification. The paper positions human-in-the-loop interaction as a mechanism for validating uncertain environmental assumptions and completing missing context.

3 Methodology

This section defines the proposed environment-aware elicitation workflow. The method is intentionally lightweight: it uses role-based agent collaboration, environmental knowledge, and targeted human clarification to improve requirement context completeness.

3.1 Framework Overview

The framework takes initial natural language requirements as input and produces context-enriched requirements plus a record of clarified environmental assumptions. Its scope is limited to eliciting and completing environmental context.

3.2 Multi-Agent Environment-Aware Workflow

The workflow coordinates three analysis roles.

3.2.1 Interviewer Agent. The interviewer agent guides the elicitation dialogue, detects vague stakeholder statements, and decides when a clarification question should be raised.

3.2.2 End-User Agent. The end-user agent examines requirements from the perspective of concrete user situations, including user goals, locations, interaction conditions, and task constraints.

3.2.3 Environment Agent. This agent identifies missing assumptions about deployment settings, devices, runtime conditions, connectivity, and operational constraints.

3.3 Environment Information Detection

This step checks whether each requirement contains enough environmental context for later analysis and implementation discussion.

3.3.1 Deployment Environment. The workflow identifies assumptions about physical, organizational, network, and platform deployment settings.

3.3.2 Device Constraints. The workflow checks whether device type, interface capability, sensing availability, battery, performance, and accessibility constraints are specified when relevant.

3.3.3 Runtime Conditions. The workflow detects missing assumptions about connectivity, latency, workload, environmental changes, availability, and failure conditions.

3.3.4 User Context Information. The workflow captures user roles, usage scenarios, location, activity state, attention level, and interaction constraints.

3.4 Knowledge-Assisted Context Understanding

Environmental knowledge is used to guide what context should be checked and what clarification questions should be asked.

3.4.1 Environment Knowledge. The method uses domain-relevant environment knowledge to identify likely deployment and runtime assumptions.

3.4.2 Context Templates. Context templates define recurring fields such as deployment setting, device, user situation, runtime condition, and operational constraint.

3.4.3 Scenario Patterns. Scenario patterns help map vague requirements to concrete usage situations without introducing unrelated functionality.

3.4.4 Prompt Strategies. Prompt strategies constrain agents to ask environment-focused questions and to report uncertainty rather than invent missing context.

3.5 Dynamic Environment Clarification

Clarification is triggered when the workflow detects environmental gaps that materially affect requirement interpretation.

3.5.1 Missing Environment Information. The method records which environmental fields are absent, ambiguous, or inconsistent with the stated user scenario.

3.5.2 Context Completion. The workflow updates requirements with stakeholder-confirmed context and separates confirmed assumptions from inferred assumptions.

3.5.3 Scenario Refinement. The workflow refines scenarios by linking requirements to concrete deployment, device, runtime, and user-context conditions.

3.6 Human-in-the-Loop Interaction

Human stakeholders review clarification questions and confirm or correct proposed environmental assumptions. The interaction is designed to improve context completeness while keeping the clarification burden manageable.

4 Experiments

The experiment section evaluates whether the proposed workflow improves environment-aware requirement elicitation. No result values are reported in this draft structure.

4.1 Research Questions

- **RQ1:** Does the workflow improve requirement context completeness?
- **RQ2:** Does knowledge-assisted analysis improve environment consistency?

- **RQ3:** Do dynamic clarification questions help stakeholders complete missing environmental information?
- **RQ4:** What is the contribution of each agent role and knowledge component?

4.2 Experimental Setup

This subsection will describe the elicitation tasks, participant or annotator protocol, model configuration, prompt setting, and evaluation procedure. The setup should focus on environment-aware elicitation rather than general text generation quality.

4.3 Datasets

The dataset should consist of natural language requirement descriptions with missing or underspecified environmental context. Candidate sources may include course project requirements, curated requirement scenarios, and manually designed environment-sensitive cases.

4.4 Baselines

Baselines should be simple and directly comparable:

- original requirements without environment-aware elicitation;
- single-agent requirement clarification;
- multi-agent workflow without environment knowledge;
- workflow without human clarification, if applicable.

4.5 Evaluation Metrics

4.5.1 Context Completeness. Measures whether deployment, device, runtime, and user-context information is sufficiently specified.

4.5.2 Environment Consistency. Measures whether elicited environmental assumptions are mutually consistent and aligned with the scenario.

4.5.3 Requirement Completeness. Measures whether the resulting requirements contain enough information for environment-aware analysis.

4.5.4 Clarification Effectiveness. Measures whether generated clarification questions help stakeholders provide missing environmental information.

4.6 Main Results

This subsection will report quantitative and qualitative findings for the research questions after experiments are conducted. Tables should distinguish confirmed results from illustrative examples.

4.7 Ablation Study

The ablation study will compare the full workflow against variants without selected agent roles, knowledge assistance, or human clarification.

4.8 Case Study

The case study will trace one requirement from its initial underspecified form through environment detection, clarification, and context-enriched requirement output.

5 Threats to Validity

This section follows standard empirical software engineering reporting practice.

5.1 Internal Validity

Potential threats include prompt sensitivity, evaluator bias, inconsistent stakeholder responses, and variability in agent outputs. Mitigations should include fixed protocols, repeated runs where appropriate, and independent annotation or review.

5.2 External Validity

The findings may depend on the selected requirement scenarios, domains, participant profiles, and environment categories. The paper should state the intended scope of generalization and avoid claims beyond environment-aware elicitation.

5.3 Construct Validity

Metrics such as context completeness and clarification effectiveness may not fully capture elicitation quality. The evaluation should define each metric clearly and align annotation rubrics with the research questions.

6 Conclusion

This paper studies environment-aware requirement elicitation as a research problem in intelligent requirements engineering. The proposed structure centers on multi-agent context understanding, knowledge-assisted requirement analysis, dynamic environment clarification, and human-in-the-loop validation. The final paper should conclude by summarizing how the workflow improves requirement context completeness and by outlining focused future work on environment-aware elicitation only.

References

[1] Ikram Darif, Ghizlane El Boussaidi, Segla Kpodjedo, and Cristiano Politowski. 2026. Controlled Natural Language for Requirements Specification: A Systematic Literature Review. *ACM Comput. Surv.* 58, 7 (Jan. 2026), 175:1–175:36. doi:10.1145/3778169

[2] Diana Robinson, Christian Cabrera, Andrew D. Gordon, Neil D. Lawrence, and Lars Mennen. 2025. Requirements Are All You Need: The Final Frontier for End-User Software Engineering. *ACM Trans. Softw. Eng. Methodol.* 34, 5 (May 2025), 141:1–141:22. doi:10.1145/3708524

A Appendix

The appendix may include the environment-context annotation rubric, example clarification questions, prompt templates, and additional case-study traces. It should not introduce new research claims that are absent from the main paper.